Technical Case Study (Candidate Brief)

Optimization + Hierarchical Forecast Reconciliation (MinT / GLS)

This case study evaluates your ability to translate business constraints into a mathematically sound optimization problem, implement it with solid numerical practices, and clearly interpret results.

Provided data file: `forecast_data_anonymized.csv`

Allowed: NumPy, SciPy, Pandas, Plotly/Matplotlib

Not allowed: black-box reconciliation libraries (e.g., hierarchicalforecast, sktime)

1) Scenario

You are the Senior Data Scientist responsible for forecasting at a platform that plans capacity using daily volume forecasts. A data quality issue has been flagged: **forecast incoherence**.

We produce independent forecasts for multiple related time series:

- **aggregate**
- **cohort_A**, **cohort_B**
- leaf cohorts: **A1**, **A2**, **B1**, **B2**

Because these forecasts are trained independently, they sometimes contradict one another. Example: the model for **aggregate** predicts 10,000 units/day, but the sum of the leaf cohort forecasts (A1 + A2 + B1 + B2) implies 11,500 units/day. Operations needs one consistent set of numbers.

2) The Hierarchy (Business Truth)

These series are related by strict accounting identities:

Root:

```
aggregate = cohort_A + cohort_B
```

Intermediate:

```
cohort_A = A1 + A2
cohort_B = B1 + B2
```

Leaves:

A1, A2, B1, B2 are mutually exclusive leaf cohorts.

3) The Data

You are provided a single file:

`forecast_data_anonymized.csv`

Columns:

- **date**: daily timestamp
- **label**: "train" or "future"
- For each series s in {aggregate, cohort_A, cohort_B, A1, A2, B1, B2}:
- `{s}_pred`: base prediction from an independent model (available in both train and future)
- `{s}_actual`: actual observed value (available in train only; blank/NA in future)

Important:

- Base predictions were generated independently for every series.
- Actuals are only available for train.

4) Your Task

Build a forecast reconciliation engine that converts the base forecasts into coherent forecasts for the future dates.

Your reconciler must:

- **Enforce coherence exactly**: aggregate = cohort_A + cohort_B; cohort_A = A1 + A2; cohort_B = B1 + B2.
- **Minimize forecast adjustments** using MinT / Generalized Least Squares (GLS).
- **Guarantee nonnegative leaf forecasts** (A1, A2, B1, B2 cannot be negative).
- Use a robust estimate of the base forecast error covariance matrix **W**, including **shrinkage**.

5) Constraints

- No black-box reconciliation libraries (do not use hierarchicalforecast, sktime, etc.).
- Allowed: NumPy, SciPy, Pandas, Plotly/Matplotlib.
- Implement the reconciliation math yourself.

6) Deliverables

A) Runnable code

A Python script or notebook that:

- Loads `forecast_data_anonymized.csv`
- Estimates W from training residuals (with shrinkage)
- Reconciles future forecasts
- Outputs `reconciled_forecasts.csv` with: date and reconciled forecasts for aggregate, cohort_A, cohort_B, A1, A2, B1, B2

B) Short memo (1-2 pages)

Your memo must cover:

- **Methodology**: hierarchy representation, W estimation + shrinkage, and nonnegativity handling.
- **Validation**: numerical checks proving coherence and nonnegativity.
- **Impact analysis**: compare base vs reconciled forecasts; interpret which series moved and why.

Required question (answer in your memo):

Why might estimating W using in-sample training residuals computed from the same training-period predictions and actuals be a poor choice? What alternative approach should be used to estimate W more realistically, and how would you implement it for time series forecasting?

Suggested plots:

- Actuals (train) + Base (future) + Reconciled (future) for each series.
- Adjustments: Delta = reconciled - base, per series.

Interpretation prompts:

- Which series moved the most and why?
- Did the base aggregate forecast pull the leaves, or did the leaves pull aggregate?
- Explain using what your estimated W implies (series reliability / correlations).

7) Evaluation Criteria

- **Correctness**: coherence is exact; nonnegativity holds; output format correct.
- **Statistical rigor**: sensible W, stable shrinkage, numerically robust.
- **Interpretability**: clear explanation of who moved and why.
- **Engineering**: clean, modular, reproducible pipeline.